

MULTIMEDIA UNIVERSITY OF KENYA

HILLARY OMONDI OCHIENG

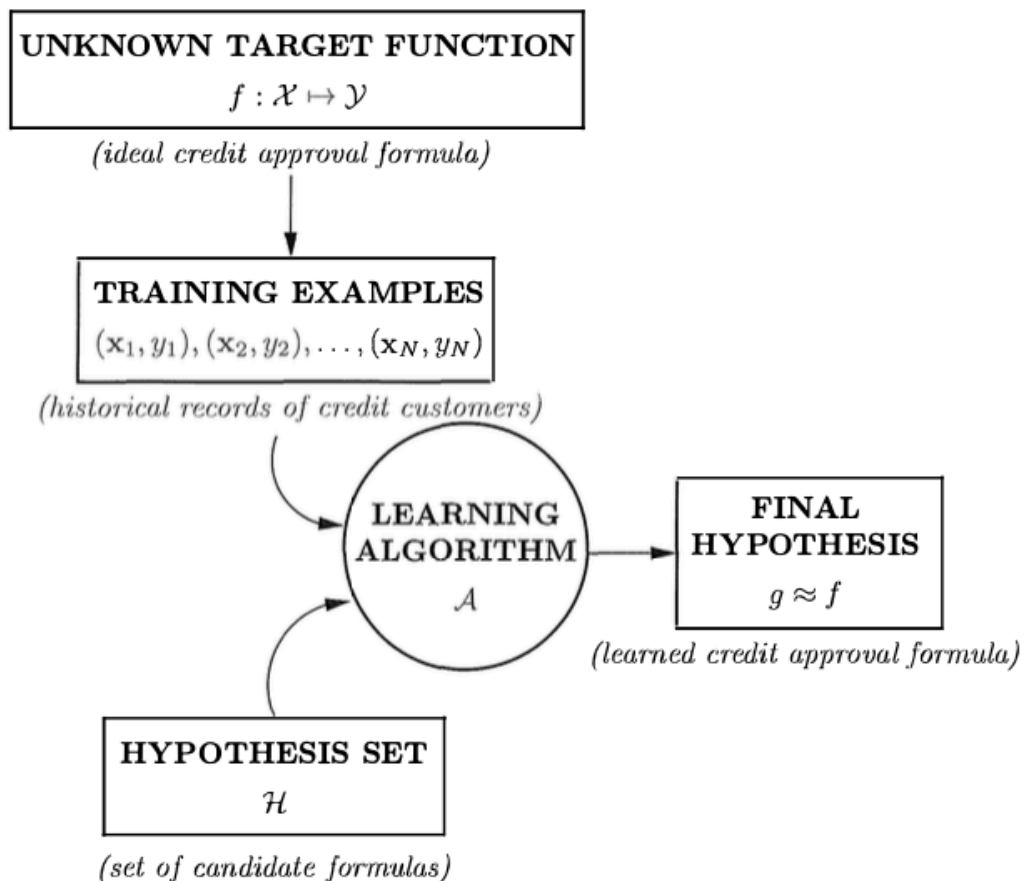
CIT-223-016/2022

**BACHELOR OF COMPUTER SCIENCE
LEARNING AND ADAPTIVE SYSTEMS
CAT 1**

Question One

(a) Describe learning components using appropriate diagrams.'

1. Input -This is the **data fed into the system**.
2. Target function ($f : X \rightarrow Y$) -This is the **true (ideal) mapping** from input to output.
3. Training Examples - These are **sample input-output pairs** used for learning.
 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
4. Learning algorithms
The procedure that:
Searches through the hypothesis set
Finds the best hypothesis based on training data
5. Final Hypothesis - The **chosen model** after training.
6. Hypothesis set - The **set of all possible models/functions** the algorithm can choose from.



(b) Initial Machine Learning aims to understand pattern classification well. Often many algorithms are used. Using a commented code, write a program to implement MNIST Dataset learning of digits 0 – 9. [10 Marks]

```
import pandas as pd
import numpy as np
from math import log2
```

```
# Dataset
data = [
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "Normal", "Weak", "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Overcast", "Mild", "High", "Strong", "Yes"],
```

```
["Overcast","Hot","Normal","Weak","Yes"],
["Rain","Mild","High","Strong","No"]
]
```

```
df = pd.DataFrame(data, columns=["Outlook","Temp","Humidity","Wind","Play"])
```

```
# Entropy function
```

```
def entropy(target):
    values, counts = np.unique(target, return_counts=True)
    probs = counts / counts.sum()
    return -sum(p * log2(p) for p in probs)
```

```
# Information Gain
```

```
def info_gain(data, feature, target="Play"):
    total_entropy = entropy(data[target])

    values, counts = np.unique(data[feature], return_counts=True)

    weighted_entropy = sum(
        (counts[i]/sum(counts)) *
        entropy(data.where(data[feature]==values[i]).dropna()[target])
        for i in range(len(values))
    )

    return total_entropy - weighted_entropy
```

```
# Compute IG for all features
```

```
for col in ["Outlook","Temp","Humidity","Wind"]:
    print(col, ":", round(info_gain(df, col), 3))
```

Question Two

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

(a) Consider the above dataset of playing decisions . Calculate information gain of each feature.

Step 1: Entropy of Dataset

Total samples = 14, Yes = 9, No = 5

$$H(S) = -[(9/14)\log_2(9/14) + (5/14)\log_2(5/14)] = 0.940$$

Step 2: Information Gain for Outlook

Sunny: (2 Yes, 3 No) $\rightarrow H \approx 0.971$

Overcast: (4 Yes, 0 No) $\rightarrow H = 0$

Rain: (3 Yes, 2 No) $\rightarrow H \approx 0.971$

$$\text{Weighted Entropy} = (5/14)*0.971 + (4/14)*0 + (5/14)*0.971 = 0.694$$

$$IG(\text{Outlook}) = 0.940 - 0.694 = 0.246$$

Step 3: Information Gain for Temperature

Hot: (2 Yes, 2 No) $\rightarrow H = 1.000$

Mild: (4 Yes, 2 No) $\rightarrow H \approx 0.918$

Cool: (3 Yes, 1 No) $\rightarrow H \approx 0.811$

Weighted Entropy ≈ 0.911

$$IG(\text{Temperature}) = 0.940 - 0.911 = 0.029$$

Step 4: Information Gain for Humidity

High: (3 Yes, 4 No) $\rightarrow H \approx 0.985$

Normal: (6 Yes, 1 No) $\rightarrow H \approx 0.592$

Weighted Entropy ≈ 0.788

$$IG(\text{Humidity}) = 0.940 - 0.788 = 0.151$$

Step 5: Information Gain for Wind

Weak: (6 Yes, 2 No) $\rightarrow H \approx 0.811$

Strong: (3 Yes, 3 No) $\rightarrow H = 1.000$

Weighted Entropy ≈ 0.892

$IG(\text{Wind}) = 0.940 - 0.892 = 0.048$

(b) Identify the most feasible to split.

Outlook - has the highest Information Gain = 0.246

This means it reduces uncertainty the most.

(c) Write a python code we can use to calculate entropy and information gain. [10 marks]

```
import pandas as pd
import numpy as np
from math import log2

# Dataset
data = [
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "Normal", "Weak", "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Overcast", "Mild", "High", "Strong", "Yes"],
    ["Overcast", "Hot", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Strong", "No"]
]

df = pd.DataFrame(data, columns=["Outlook", "Temperature", "Humidity", "Wind", "Play"])

# Entropy function
def entropy(target):
    values, counts = np.unique(target, return_counts=True)
    probs = counts / counts.sum()
    return -sum(p * log2(p) for p in probs)

# Information Gain function
def information_gain(df, feature, target="Play"):
    total_entropy = entropy(df[target])
```

```

values, counts = np.unique(df[feature], return_counts=True)

weighted_entropy = sum(
    (counts[i] / sum(counts)) *
    entropy(df[df[feature] == values[i]][target])
    for i in range(len(values))
)

return total_entropy - weighted_entropy

# Compute for each feature
for feature in ["Outlook", "Temperature", "Humidity", "Wind"]:
    print(f"{feature}: {round(information_gain(df, feature), 3)}")

```